

Tecnologías utilizadas en FastKote

Proyecto: FastKote - Sistema de generación de cotizaciones

Cuaderno: Construcción e implementación

Versión analizada: fastkote-fullstack-layered-v2 (2).zip

1. Resumen tecnológico

FastKote fue implementado como una aplicación web fullstack bajo una arquitectura cliente-servidor. El frontend está construido con React, Vite y TypeScript; el backend usa Node.js, Express y TypeScript; la persistencia se realiza en PostgreSQL mediante Prisma ORM y scripts SQL.

Esta selección tecnológica se alinea con las restricciones del SRS V1.0, donde el sistema debe funcionar como aplicación web accesible desde navegador, comunicarse por HTTP/HTTPS y usar una base de datos relacional para conservar la integridad de clientes, cotizaciones, paquetes, servicios, insumos y usuarios.

2. Tecnologías del frontend

Tecnología	Uso en FastKote	Relación RF/CU
React	Construcción de interfaces de usuario.	RF-01, RF-02, RF-03, RF-06.
Vite	Servidor de desarrollo y empaquetado del frontend.	Soporte técnico del cliente web.
TypeScript	Tipado estático para componentes, APIs y estado.	Mejora mantenibilidad de los módulos.
Fetch API	Consumo de endpoints REST del backend.	Comunicación cliente-servidor.
CSS	Estilos globales, layout, modales y páginas.	Interfaces de usuario RF-UI.

Estructura frontend relevante

```
frontend/src/  
├─ api/  
├─ auth/  
├─ components/  
├─ pages/  
└─ styles/
```

Páginas construidas

Página / componente	Función	RF/CU relacionado
LandingPage.tsx	Vista pública inicial del sistema.	Presentación general del sistema.
LoginScreen.tsx	Inicio de sesión.	RF-01 / CU-SGC-Lo-01.
DashboardPage.tsx	Panel principal posterior al login.	Acceso a módulos por rol.
ClientsPage.tsx	Administración de clientes.	RF-03 / CU-SGC-Lo-04.
EmployeesPage.tsx	Administración de empleados y roles.	RF-06 / CU-SGC-Lo-05.
QuotesPage.tsx	Gestión de cotizaciones.	RF-02 / CU-SGC-Lo-03.
CalendarPage.tsx	Consulta de calendario.	CU-SGC-L1-17.
ConstructionPage.tsx	Página temporal para registro público.	Funcionalidad futura.

3. Tecnologías del backend

Tecnología	Uso en FastKote	Relación RF/CU
Node.js	Entorno de ejecución del backend.	Soporte general del sistema.
Express	API REST y enrutamiento HTTP.	RF-01, RF-02, RF-03, RF-06.
TypeScript	Tipado y organización del backend.	Mantenibilidad y calidad.
Zod	Validación de datos de entrada en handlers.	Validaciones de CU.
Prisma ORM	Acceso a PostgreSQL desde repositorios.	Persistencia relacional.
PostgreSQL	Base de datos relacional.	RF-01, RF-02, RF-03, RF-04, RF-06.
JWT	Autenticación basada en token.	RF-01 / CU-SGC-Lo-01.
bcryptjs	Hash y comparación de contraseñas.	Seguridad de acceso.
PDFKit	Generación de PDF de cotización.	CU-SGC-L1-18.
CORS	Permite comunicación frontend-backend.	Comunicación cliente-servidor.
dotenv	Configuración por variables de entorno.	Portabilidad y configuración.

4. Dependencias principales del backend

Archivo: backend/package.json

```
@prisma/client
bcryptjs
cors
dotenv
express
jsonwebtoken
```

pdfkit
zod

Uso por dependencia

Dependencia	Uso
@prisma/client	Cliente ORM para consultar y modificar PostgreSQL.
bcryptjs	Hash y validación de contraseñas.
cors	Habilita solicitudes desde el frontend.
dotenv	Carga variables de entorno.
express	Servidor API REST.
jsonwebtoken	Firma y verificación de JWT.
pdfkit	Construcción del PDF de cotización.
zod	Validación de payloads en casos de uso.

5. Dependencias principales del frontend

Archivo: frontend/package.json

@vitejs/plugin-react
vite
typescript
react
react-dom

Uso por dependencia

Dependencia	Uso
react	Construcción de componentes de interfaz.
react-dom	Renderizado de React en el navegador.
vite	Desarrollo y build del frontend.
typescript	Tipado de componentes y servicios.
@vitejs/plugin-react	Integración React con Vite.

6. Tecnologías de base de datos

Tecnología	Uso
PostgreSQL	Motor de base de datos relacional.
Prisma ORM	Mapeo objeto-relacional desde TypeScript.
SQL	Creación manual de tablas y datos base.
Docker Compose	Levantamiento del contenedor PostgreSQL local.

Base configurada en desarrollo:

Base: fastkote_db

Usuario: fastkote_user

Clave: fastkote_pass

Puerto: 5432

Archivos relevantes:

docker-compose.yml

backend/prisma/schema.prisma

```
database/01_schema.sql
database/02_seed.sql
```

7. Tecnologías y patrones de diseño

La construcción usa dos patrones principales:

Patrón	Ubicación	Uso	RF/CU relacionado
Mediator	backend/src/application/mediator/Mediator.ts	Desacopla controladores de handlers.	RF-01, RF-02, RF-03, RF-06.
Strategy	backend/src/application/quotes/strategies/	Separa cálculos y reglas de estado.	RF-02, CU-SGC-L1-11, CU-SGC-L1-12, CU-SGC-L1-15.

8. Variables de entorno relevantes

Archivo: backend/src/shared/config/env.ts

```
PORT
DATABASE_URL
JWT_SECRET
JWT_EXPIRES_IN
FRONTEND_URL
TAX_RATE
WHATSAPP_API_URL
WHATSAPP_API_TOKEN
```

Descripción

Variable	Uso
PORT	Puerto del backend, por defecto 4000.
DATABASE_URL	Cadena de conexión a PostgreSQL.
JWT_SECRET	Clave para firmar tokens JWT.
JWT_EXPIRES_IN	Tiempo de expiración del token.
FRONTEND_URL	URL permitida por CORS.
TAX_RATE	Tasa de IVA usada en cálculos de cotización.
WHATSAPP_API_URL	URL del proveedor externo de WhatsApp.
WHATSAPP_API_TOKEN	Token del proveedor de WhatsApp.

9. Relación tecnología - requisito

Requisito / CU	Tecnología que lo soporta
RF-01 / CU-SGC-Lo-01 Autenticar Usuario	Express, JWT, bcryptjs, Prisma, PostgreSQL, React.
RF-02 / CU-SGC-Lo-03 Administrar Cotización	React, Express, Mediator, Strategy, Prisma, PostgreSQL.
CU-SGC-L1-11 Generar Cotización	Strategy de cálculo, Prisma, PostgreSQL.
CU-SGC-L1-15 Actualizar Estado Cotización	Strategy de estado, tabla <code>event_reservations</code> .
CU-SGC-L1-17 Consultar Calendario de Eventos	React, Express, Prisma, PostgreSQL.

Requisito / CU	Tecnología que lo soporta
CU-SGC-L1-18 Generar PDF de Cotización	PDFKit, Express.
CU-SGC-L1-19 Enviar Cotización por WhatsApp	WhatsAppGateway, variables de entorno.
RF-03 / CU-SGC-Lo-04 Administrar Cliente	React, Express, Prisma, PostgreSQL.
RF-06 / CU-SGC-Lo-05 Administrar Empleados	React, Express, JWT, roles, Prisma, PostgreSQL.

10. Conclusión

Las tecnologías seleccionadas permiten construir FastKote como una aplicación web modular, mantenible y alineada con el alcance de cotizaciones definido en el SRS. React y Vite resuelven la interfaz de usuario; Express y TypeScript organizan la API; Prisma y PostgreSQL garantizan persistencia relacional; JWT y bcryptjs cubren la seguridad de acceso; PDFKit permite generar cotizaciones formales; y el gateway de WhatsApp mantiene desacoplada la integración externa.